

Large scale data management project

Members: Trasolini Gabriele 1858396, Demuro Giulio 1920126

Type of project: use of a noSQL tool (i.e. MongoDB) and use of an Information integration tool (i.e. KNIME). Our goal is to cover both parts with the same project.

Datasets:

1. dataset.csv (The Baseline) This is the **primary baseline dataset** for the project. It contains over 114,000 unique music tracks. It provides the structural backbone of the song-level integration, including unique identifiers (track_id), administrative metadata (artists, album_name, track_name), and high-dimensional technical audio features such as danceability, energy, loudness, tempo, and acousticness;

2. test.csv/train.csv(they share the same schema): The companion to the training dataset, used for **model evaluation and validation**. It shares the same schema as train.csv (excluding the target class) and is used to verify the accuracy of the integration and classification logic on unseen data;

3.all_songs_data.csv A **historical and semantic enrichment source** that links Billboard chart data with content from Genius. It provides significant qualitative value by adding Lyrics, Release Date, Writers, and historical Rank. It is instrumental in transforming technical audio data into a comprehensive knowledge base that includes songwriting context and chronological information;

4. spotify_millsongdata.csv A **large-scale text repository** focused on song lyrics. It contains artist names, song titles, and the full textual content of tracks. In the integration workflow, it serves as a high-volume source for text mining and as a backup to populate missing lyrics fields in the baseline dataset;

5. artists-data.csv An **artist-centric metadata source** that shifts the focus from individual tracks to the creators. It includes fields for Genres, the total number of Songs published by the artist, and a Popularity score. This file is used to build a separate "Artists" entity within the relational data model;

6. artists.csv A **global artist metrics dataset** containing platform-specific information from Spotify. Key attributes include unique id, total followers count, and main_genre classification. This file provides the quantitative "social impact" metrics needed to enrich the artist profiles in the final database.

Description of the work: we are going to develop the project using Docker as a development environment.

Our goal is to deploy a complete web application (through containers) called Vibe, implemented with PHP and HTML (+ CSS), in order to cover a possible real case scenario where users will be able to register and subsequently log in.

On the homepage they will find several accessible sections and specific actions to be performed, related to our datasets. We think that this approach should represent a real-world scenario.

The web application is organized by splitting its components into separate containers to ensure modularity and ease of management.

The containers are defined and created using Docker Compose through a YAML configuration file, which allows the entire multi-container application to be managed and started with a single command.

One container is dedicated to handling the database layer, running MongoDB. This container is responsible for data storage, retrieval, and management, and it operates independently from the rest of the application.

A second container manages both the frontend and backend of the application. It includes the user interface built with HTML as well as the server-side logic implemented in PHP. This container handles client requests, processes data, and communicates with the MongoDB container when database operations are required. This separation into distinct containers improves scalability, simplifies maintenance, and allows each component to be updated or deployed independently.

Before explaining the values of the global schema, we also report the fields of each dataset defined in the proposal part above.

dataset.csv: track_id,artists,album_name,track_name,popularity,duration_ms,explicit,danceability,energy,key,loudness,mode,speechiness,acousticness,instrumentalness,liveness,valence,tempo,time_signature,track_genre;

train.csv/test.csv: Artist Name,Track

Name,Popularity,danceability,energy,key,loudness,mode,speechiness,acousticness,instrumentalness,liveness,valence,tempo,duration_in min/ms,time_signature,Class(is only for train);

all_song_data.csv:

Album,Album URL,Artist,Featured Artists,Lyrics,Media,Rank,Release Date,Song Title,Song URL,Writers,Year

spotifymillsongdata.csv: artist,song,link,text

The integration process combines multiple heterogeneous music datasets into a unified global schema.

Records are matched primarily through normalized combinations of Song_Title and Artist_Name fields. Since different datasets may contain inconsistencies such as alternative artist naming, featured artists, duplicated songs, or formatting differences, preprocessing operations will be applied.

Global Schema

Track_ID:Primary integrated identifier derived from dataset.csv track_id when available.

Song_Title: Title of the song.

Artist_Name: Name of principal artist.

Featured_Artists (Type: List/Array) list of featured artists collaborating on the track.

Album_name:the name of the album that contains that specified song

Technical_Metrics: An object/group: duration_ms, explicit, danceability ,energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, time_signature,

Full_Lyrics: full integrated textual lyrics of the song.

Writers: The authors of the song.

Genre: genre assigned through integration of track-level and artist-level genre information

Popularity_Index: popularity score associated with the artist(0-100).

Release_Date: release date of the song(Y-M-D).

Rank: chart ranking position when available across integrated datasets.

Year_Rank: year associated with the ranking information.

Album_url: Url for the album(if it is present)

Song_url: Url for the song(if it is present)

Media_link: link youtube for the song(if it is present)

Then we have the collection for the artists, as above we bring the fields from the following datasets:

artist.csv:id,name,followers,popularity,genres,main_genre

artist-data.csv:Artist,Genres,Songs,Popularity,Link

Artist entities are matched through normalized artist names and cleaned metadata in order to reduce inconsistencies across datasets.

Global schema:

Artist_ID: unified identifier for the artist, derived from artist.csv when available

Name: The name of the artist;

Followers: total number of followers associated with the artist.

Popularity: the popularity of the artist;

Genres: list of musical genres associated with the artist.

Main_Genre: the primary musical genre associated with the artist.

Number_of_songs: total number of songs published by the artist.

Link:URL associated with the artist profile or external reference page.

